TECHNICAL DOCUMENTATION

RegulareEdge

Regulatory Reporting & Compliance Management Platform

A Spring Boot microservices backend with a React single-page frontend. Complete reference: architecture, all services, data model, APIs, security, and the frontend.

Backend · 8 services

Java 21 · Spring Boot 3.2.0 · Spring Cloud 2023.0.0 · MySQL 8

Frontend

React 18 · React Router 6 · Axios · Vite 5

1. Introduction & Purpose

RegulareEdge is a Regulatory Reporting & Compliance Management Platform. Regulated businesses — banks, financial firms and similar — must periodically submit official reports (“regulatory returns”) to government regulators such as RBI or SEBI. These reports must be correct, complete and submitted before a deadline; late or wrong submissions lead to penalties.

The platform moves this entire process onto one system instead of scattered emails and spreadsheets. It coordinates the people involved (admins, compliance officers, data owners, approvers, auditors and legal officers), tracks deadlines, collects and certifies data, routes returns for approval, records penalties, and keeps a full audit trail of every action.

Advantages

- **No missed deadlines** — every due date is tracked, with reminder notifications.
- **Clear accountability** — the system records who prepared, certified and approved each item.
- **Audit-ready** — every significant action is written to an audit log for inspections.
- **Faster & consistent** — a guided step-by-step flow with validation replaces manual copy-paste.
- **Central visibility** — dashboards show the status of returns, approvals and penalties in one place.

Problems it solves (before vs after)

Problem before	How RegulareEdge solves it
Reports tracked in Excel / email — things get missed	One central system with deadlines and reminders
No proof of who did what	Every action stored in an audit log
Late submissions lead to penalties	Deadline tracking + notifications
Data collected manually, no review	Structured flow: request → provide → certify → approve
No control over who can do what	Role-based access — each person sees only their own screens

2. Roles

There are six roles. Access is enforced end-to-end: the frontend hides screens a role should not see, and every backend API is protected with Spring Security @PreAuthorize so the correct role is required even if the API is called directly.

Role	Backend enum	Responsibility
Admin	ADMIN	Sets up master data: regulators, obligations, deadlines (calendar), and creates all users.
Compliance Officer	CO	Requests data from data owners, creates the returns, prepares them for approval.
Data Owner	DATA_OWNER	Owns the business data. Uploads the required document (PDF) and certifies it is correct.
Chief Compliance Officer	CCO	Senior approver. Reviews the return and its PDF and gives final approval.
Internal Auditor	AUDITOR	Independently reviews obligations and records audit ratings; read-only views.
Legal Officer	LEGAL	Handles penalties — records them and tracks payment / dispute status.

Note: the frontend uses long role names (COMPLIANCE_OFFICER, INTERNAL_AUDITOR, LEGAL_OFFICER) which the API layer translates to the backend's short enum (CO, AUDITOR, LEGAL). ADMIN, DATA_OWNER and CCO are identical on both sides.

3. System Architecture

The backend is composed of eight Spring Boot services: two infrastructure services (service discovery and central configuration), one edge gateway, and five business services, each owning its own MySQL database. The React frontend talks only to the API gateway.

3.1 Services & ports

Service	Port	Type	Database	Responsibility
eureka-server	8761	Infra	—	Service discovery / registry
config-server	8888	Infra	—	Central configuration (native, from config-repo)
api-gateway	9080	Edge	—	Single entry point, JWT gate, routing, CORS
auth-service	8081	Business	authdb	Users, login, JWT issue, roles, registration
compliance-core-service	8082	Business	compliancecoredb	Regulators, obligations, calendar, data requests, certification, PDF
return-management-service	8083	Business	returnmanagementdb	Returns + documents, schedules, submissions, correspondence, penalties
audit-analytics-service	8084	Business	auditanalyticsdb	Audit reviews, control evidence, compliance reports, CCO analytics
notification-service	8085	Business	notificationdb	In-app notifications (create, fetch, unread, mark-read)

3.2 How the pieces fit together

- **Service discovery (Eureka)** — every service registers with Eureka at http://localhost:8761/eureka/. Services call each other by name (lb://service-name), load-balanced, with no hard-coded URLs.
- **Central config (Config Server)** — runs in native mode and serves per-service YAML from config-server/config-repo/. A shared application.yml provides the JWT secret, Eureka settings, logging and Resilience4j defaults inherited by all services.
- **API Gateway (Spring Cloud Gateway)** — the only endpoint the browser uses (http://localhost:9080). It validates the JWT on every request (except public paths), applies CORS, adds a correlation id for tracing, logs each request, and routes by path prefix to the right service.
- **Inter-service calls (OpenFeign)** — services call one another via Feign clients (e.g. compliance-core validates a user against auth-service and sends notifications to notification-service). The caller's JWT and correlation id are propagated on these calls.
- **Resilience (Resilience4j)** — Feign calls are wrapped with circuit breaker + retry, and are fail-open: if a downstream service is unavailable, a safe fallback is used and a 'degraded' flag may be surfaced instead of failing the request.

3.3 Gateway routes

Route (path prefix)	Target service
/auth/**, /users/**	auth-service
/regulators/**, /obligations/**, /calendar/**, /data-requests/**, /certifications/**	compliance-core-service
/returns/**, /schedules/**, /submissions/**, /correspondence/**, /penalties/**	return-management-service
/audit-reviews/**, /control-evidence/**, /analytics/**	audit-analytics-service
/notifications/**	notification-service
/{service}/v3/api-docs (StripPrefix=1)	aggregated Swagger / OpenAPI per service

3.4 Security model

- **Authentication** — JWT (JSON Web Token). auth-service issues an access token (1 hour) and a refresh token (7 days) on login. The token is signed with a shared HMAC-SHA secret (HS256) held in the config server, so every service can validate it.
- **Token claims** — subject = user email, plus a custom 'role' claim. Services map the role to a Spring authority ROLE_<role>.
- **Two layers of checks** — the gateway rejects requests without a valid token; each service additionally re-validates the token and enforces @PreAuthorize on the endpoint.
- **Passwords** — stored BCrypt-hashed in auth-service. Sessions are stateless (no server session); CSRF disabled because there are no cookies.
- **Public paths** — /auth/**, Swagger UI, /v3/api-docs and /actuator are open; everything else requires authentication.

Environment note (development): the JWT secret and MySQL credentials (root/root) are committed in the config repo, ddl-auto is 'update', and databases auto-create. These are development conveniences that would be externalised (secrets manager, migrations, least-privilege DB users) for production.

4. Technology Stack

4.1 Backend

Technology	Version	Purpose
Java	21	Language / runtime
Spring Boot	3.2.0	Core application framework
Spring Cloud	2023.0.0	Microservices BOM
Spring Cloud Netflix Eureka	via 2023.0.0	Service discovery (server + clients)
Spring Cloud Config	via 2023.0.0	Centralised configuration
Spring Cloud Gateway	via 2023.0.0	API gateway (reactive / WebFlux)
Spring Cloud OpenFeign	via 2023.0.0	Declarative service-to-service REST
Spring Security	6.2.x (via Boot)	Authentication + role-based authorization
Spring Data JPA / Hibernate	6.4.x (via Boot)	ORM / persistence
JJWT (io.jsonwebtoken)	0.12.5	JWT create / verify
springdoc-openapi	2.3.0	Swagger UI / OpenAPI docs
Resilience4j	2.2.0	Circuit breaker, retry, rate limiter
MySQL Connector/J	8.2.0 (via Boot)	MySQL JDBC driver
MySQL Server	8.0	Relational database (one per service)
H2	test scope	In-memory DB for tests
Maven	—	Build tool (spring-boot-maven-plugin)

4.2 Frontend

Technology	Version	Purpose
React	18.2.0	UI library
React DOM	18.2.0	DOM rendering
React Router DOM	6.22.0	Routing + route guards
Axios	1.6.0	HTTP client to the API gateway
Vite	5.x	Build tool / dev server
@vitejs/plugin-react	4.2.0	React support in Vite
CSS	custom (index.css)	Styling — no CSS framework

5. Backend Services

5.1 eureka-server (Service Discovery)

Port: 8761

Main class: EurekaServerApplication — @SpringBootApplication, @EnableEurekaServer

Standalone registry: does not register with or fetch from itself (register-with-eureka / fetch-registry = false). Self-preservation is disabled for fast dev turnaround, so dead instances are dropped quickly. It is self-contained and does not import from the config server.

5.2 config-server (Central Configuration)

Port: 8888

Main class: ConfigServerApplication — @SpringBootApplication, @EnableConfigServer

Runs with the 'native' profile, serving configuration files from the local config-server/config-repo folder (not a Git backend). Registers with Eureka. A shared application.yml is inherited by every client service and defines:

- Eureka client settings (defaultZone, prefer-ip-address, 10s lease renewal).
- JWT: shared secret, access-token expiry 3,600,000 ms (1h), refresh-token expiry 604,800,000 ms (7d).
- Resilience4j defaults (circuit breaker window 10 / 50% failure threshold, retry 3 × 500ms, time limiter 2s, rate limiter 20/s).
- Logging with a correlation-id (MDC) pattern; actuator exposure health/info/refresh/circuitbreakers.

Per-service files provide each service's port, MySQL datasource, JPA settings, Swagger paths, and (where needed) multipart limits and file.upload-dir. compliance-core and return-management set 15MB multipart limits and an upload directory for PDFs.

5.3 api-gateway (Edge Router)

Port: 9080

Main class: ApiGatewayApplication — @SpringBootApplication (Eureka client via classpath)

Type: Reactive (Spring Cloud Gateway on WebFlux)

The single entry point for the browser. Key components:

- **JwtAuthenticationGlobalFilter** — validates the Bearer token's signature and expiry (shared HMAC secret) on every request. Public paths (/auth/, swagger, /v3/api-docs, /webjars, /actuator) bypass the check. Returns structured 401 JSON on missing / expired / invalid tokens. The Authorization header is passed through so services can re-validate.
- **CorrelationIdFilter** — ensures every request/response carries an X-Correlation-Id (generated if absent) for end-to-end tracing.
- **RequestLoggingFilter** — logs method, path, correlation id, status and duration for each request.
- **Global CORS + DedupeResponseHeader** — permissive CORS with duplicate CORS-header de-duplication; a WebFlux JSON error handler replaces the whitelabel page.

Routing and OpenAPI aggregation are defined in config-repo/api-gateway.yml (see 3.3). Discovery-locator is disabled — only the explicit routes are exposed.

5.4 auth-service (Identity)

Port: 8081

Main class: AuthServiceApplication — @SpringBootApplication, @EnableConfigurationProperties(JwtProperties)

Entities

Field	Type	Notes
User (users)	—	userId(int @Id), name, email(unique), password(BCrypt), phone, businessUnitId(int), role(UserRole), status(UserStatus)
RefreshToken	—	id(Long), userId(int), token(unique,512), expiryDate(LocalDateTime), revoked(boolean)

Field	Type	Notes
PasswordResetToken	—	id(Long), userId(int), token(UUID,unique), expiryDate, used(boolean)
AuditLog (audit_logs)	—	auditId(Long), userId(Integer), action, entityType, recordId, timestamp

Enums

UserRole: ADMIN, CO, DATA_OWNER, AUDITOR, LEGAL, CCO

UserStatus: ACTIVE, INACTIVE, LOCKED

Endpoints

Method	Path	Role	Request	Response	Description
POST	/auth/register	public*	RegisterRequest	UserResponse	Register a user. Only ADMIN can self-register; other roles require an authenticated ADMIN caller.
POST	/auth/login	public	LoginRequest	LoginResponse	Authenticate; issue access + refresh tokens.
POST	/auth/refresh	public	RefreshTokenRequest	TokenResponse	Issue a new access token from a valid refresh token.
POST	/auth/forgot-password	public	ForgotPasswordRequest	MessageResponse	Create a 15-min password-reset token.
POST	/auth/reset-password	public	ResetPasswordRequest	MessageResponse	Reset password using the token.
GET	/users	ADMIN	Pageable	Page<UserResponse>	List all users (paged).
GET	/users/me	authenticated	—	UserResponse	Current user, resolved from the token's email.
GET	/users/role/{role}	authenticated	path+Pageable	Page<UserResponse>	List users of a role (for assignment dropdowns).
PUT	/users/status	ADMIN	UpdateUserStatusRequest	UserResponse	Activate / deactivate a user by email.
DELETE	/users	ADMIN	DeleteUserByEmailRequest	MessageResponse	Delete a user by email.
GET	/audit-logs	ADMIN, AUDITOR	filters+Pageable	Page<AuditLogResponse>	Search audit logs (userId/ entityType/action/date range).

** /auth/** is public at the filter level; the ADMIN-only rule for register is enforced in code by inspecting the caller's authorities.*

Key logic

- **Login** — BCrypt password check; on success generates access+refresh JWT (claims: email subject + role), stores the refresh token, and audit-logs LOGIN_SUCCESS (failures log LOGIN_FAILED).
- **Registration** — rejects duplicate email; new users default to status ACTIVE. Admin-only self-registration bootstraps the first admin; all other users are created by an admin.
- **JWT (JwtUtil)** — HMAC-SHA signing from the shared secret; validates signature + expiry; extracts email and role. A JwtAuthenticationFilter sets the authority ROLE_<role> from the token.
- **Security** — stateless, CSRF off, BCryptPasswordEncoder, method security (@PreAuthorize), custom 401 JSON entry point.

5.5 compliance-core-service (Regulators, Obligations, Data Collection)

Port: 8082

Main class: ComplianceCoreServiceApplication — @SpringBootApplication, @EnableFeignClients, @EnableConfigurationProperties(JwtProperties)

Entities

Field	Type	Notes
Regulator	—	regulatorId(int @Id), name, jurisdiction, regulatoryDomain(String), contactDetails, status(RegulatorStatus)
RegulatoryObligation	—	obligationId(int), regulator(@ManyToOne), returnName, returnCode, frequency(Frequency), submissionMode, returnTemplateRef, ownerId(int), status(ObligationStatus)
ComplianceCalendar	—	calendarId(int), obligation(@ManyToOne), dueDate(LocalDate), dataCutOffDate(LocalDate), reminderDays(int), status(CalendarStatus)
DataCollectionRequest	—	requestId(int), calendar(@ManyToOne), dataOwnerId(int), dataDescription, dataCutOffDate, submissionDeadline, status(DataRequestStatus)
DataCertification	—	certificationId(int), request(@ManyToOne), certifiedById(int), certificationDate, certificationStatement, status(CertificationStatus)
DataSubmissionDocument	—	documentId(int), request(@ManyToOne), fileName, storedFileName(UUID), contentType, fileSizeBytes(long), uploadedById(int), uploadedDate, status(DocumentStatus)
AuditLog	—	auditId(Long), userId, action, entityType, recordId, timestamp

Enums

RegulatorStatus: ACTIVE, INACTIVE

ObligationStatus: ACTIVE, INACTIVE, ARCHIVED

Frequency: DAILY, WEEKLY, MONTHLY, QUARTERLY, HALF_YEARLY, ANNUALLY, AD_HOC

CalendarStatus: PENDING, DUE, OVERDUE, COMPLETED

DataRequestStatus: PENDING, IN_PROGRESS, SUBMITTED, COMPLETED, OVERDUE

CertificationStatus: PENDING, CERTIFIED, REJECTED

DocumentStatus: SUBMITTED, FORWARDED

Endpoints

Method	Path	Role	Request	Response	Description
POST	/regulators/add	ADMIN	RegulatorRequest	RegulatorResponse	Create a regulator (defaults ACTIVE).
GET	/regulators/getAll	ADMIN, CO	—	List<RegulatorResponse>	All regulators.
GET	/regulators/getActive	authenticated	—	List<...>	Only ACTIVE regulators.
DELETE	/regulators/delete	ADMIN	DeleteRegulatorRequest	MessageResponse	Soft-delete (INACTIVE); blocked if active obligations exist.
POST	/obligations/add	ADMIN	RegulatoryObligationRequest	...Response	Create an obligation (regulator must be ACTIVE).
GET	/obligations/getAll	ADMIN	—	List<...>	All obligations.
GET	/obligations/getActive	authenticated	—	List<...>	ACTIVE obligations.
GET	/obligations/byRegulator/{id}	authenticated	path	List<...>	Obligations for a regulator.
GET	/obligations/myObligations/{ownerId}	CO	path	List<...>	Obligations owned by a CO.
PUT	/obligations/updateStatus	ADMIN	UpdateObligationStatusRequest	...Response	Change obligation status.
POST	/calendar/add	ADMIN	ComplianceCalendarRequest	...Response	Create a deadline (defaults PENDING).
GET	/calendar/getAll	ADMIN	—	List<...>	All calendar entries.
GET	/calendar/upcoming	authenticated	—	List<...>	Due within the next 30 days.
GET	/calendar/overdue	authenticated	—	List<...>	Due date before today.
GET	/calendar/byObligation/{id}	authenticated	path	List<...>	Deadlines for an obligation.

Method	Path	Role	Request	Response	Description
PUT	/calendar/updateStatus	ADMIN	UpdateCalendarStatusRequest	...Response	Change calendar status.
POST	/data-requests/create	CO	DataCollectionCreateRequest	...Response	Raise a data request (defaults PENDING); notifies the data owner.
GET	/data-requests/myRequests/{ownerId}	DATA_OWNER	path	List<...>	Requests assigned to a data owner.
PUT	/data-requests/updateStatus	DATA_OWNER	UpdateDataRequestStatusRequest	...Response	Update request status.
GET	/data-requests/getAll	ADMIN, CO, CCO	—	List<...>	All data requests.
POST	/data-requests/{id}/documents	DATA_OWNER	multipart file + uploadedById	DataSubmissionDocumentResponse	Upload a PDF against a request (PDF only).
GET	/data-requests/{id}/documents	authenticated	path	List<...Response>	List documents for a request.
GET	/data-requests/documents/{docId}/download	authenticated	path	PDF (Resource)	Stream / download the PDF (inline).
POST	/certifications/add	DATA_OWNER	DataCertificationRequest	...Response	Certify a request; auto-sets the request to COMPLETED.
GET	/certifications/byRequest/{id}	authenticated	path	List<...>	Certifications for a request.
GET	/audit-logs	ADMIN, AUDITOR	filters+Pageable	Page<AuditLogResponse>	Search audit logs.

Key logic

- **Certification auto-completes the request** — creating a certification saves it as CERTIFIED and sets the parent DataCollectionRequest to COMPLETED.
- **Notifications on create** — obligation, calendar and data-request creation each send a fire-and-forget notification (via Feign) to the relevant owner / data owner.
- **User validation via Feign** — owner/uploader ids are validated against auth-service (circuit-breaker + retry, fail-open).
- **PDF handling** — files are validated as PDF, stored on disk under uploads/compliance-core-service/data-submissions as <UUID>.pdf, with a metadata row; download streams the file with the original filename. 15MB multipart limit.
- **Auditing & tracing** — each service method writes an audit-log row (fail-open); a logging aspect records entry/exit/timing for controllers and services.

5.6 return-management-service (Returns, Submissions, Penalties)

Port: 8083

Main class: ReturnManagementServiceApplication — @SpringBootApplication, @EnableFeignClients, @EnableConfigurationProperties(JwtProperties)

Entities

Field	Type	Notes
RegulatoryReturn	—	returnId(int), obligationId, calendarId, reportingPeriod, preparedById, totalSchedules, submissionDeadline, status(String), documentFileName, documentStoredName, documentUploadedById, documentUploadedDate
ReturnSchedule	—	scheduleId(int), regulatoryReturn(@ManyToOne), scheduleName, dataContent(TEXT), validatedById, status(String)
SubmissionRecord	—	submissionId(int), regulatoryReturn(@ManyToOne), submittedById, submissionDateTime, submissionMode, acknowledgementRef, status(SubmissionStatus)
RegulatoryCorrespondence	—	correspondenceId(int), regulatorId, type(CorrespondenceType), receivedDate, subject, responseDueDate, assignedToId, status(CorrespondenceStatus)

Field	Type	Notes
PenaltyProceeding	—	penaltyId(int), correspondence(@ManyToOne), obligationId, penaltyType, penaltyAmount(double), disputeStatus, paidDate, status(PenaltyStatus)
AuditLog	—	auditId(Long), userId, action, entityType, recordId, timestamp

Enums

ReturnStatus: DRAFT, IN_PROGRESS, PENDING_CCO_APPROVAL, APPROVED, SUBMITTED, REJECTED (status column is a validated String)

ScheduleStatus: DRAFT, VALIDATED, REJECTED

SubmissionStatus: PENDING, SUCCESS, FAILED

CorrespondenceType: NOTICE, QUERY, WARNING, SHOW_CAUSE

CorrespondenceStatus: OPEN, RESPONDED, CLOSED

PenaltyStatus: OPEN, DISPUTED, PAID, WAIVED

Endpoints

Method	Path	Role	Request	Response	Description
POST	/returns/create	CO	RegulatoryReturnRequest	RegulatoryReturnResponse	Create a return (status DRAFT); validates obligation/calendar/preparer.
GET	/returns/myReturns/{userId}	CO	path	List<...>	Returns prepared by the user.
GET	/returns/getByStatus?status=	CO, CCO	query	List<...>	Filter returns by status.
GET	/returns/getAll	ADMIN, CCO	—	List<...>	All returns.
GET	/returns/pendingApprovals	CCO	—	List<...>	Returns in PENDING_CCO_APPROVAL.
PUT	/returns/updateStatus	CO, CCO	UpdateReturnStatusRequest	...Response	Change return status.
GET	/returns/dashboard-summary	CCO	—	CcoDashboardResponse	CCO dashboard aggregation.
POST	/returns/{id}/document	CO	multipart file + uploadedById	...Response	Upload the return's PDF (PDF only).
GET	/returns/{id}/document/download	CO, CCO, ADMIN	path	PDF (Resource)	Download the return's PDF.
POST	/schedules/add	CO	ReturnScheduleRequest	...Response	Add a schedule to a return (DRAFT).
GET	/schedules/byReturn/{id}	authenticated	path	List<...>	Schedules for a return.
PUT	/schedules/validate/{id}?validatedById=	CO	path+query	...Response	Mark schedule VALIDATED.
POST	/submissions/record	CO	SubmissionRecordRequest	...Response	Record a submission (SUCCESS if ack ref present).
GET	/submissions/byReturn/{id}	authenticated	path	List<...>	Submissions for a return.
POST	/correspondence/create	LEGAL, CO	RegulatoryCorrespondenceRequest	...Response	Create correspondence (OPEN).
GET	/correspondence/byRegulator/{id}	LEGAL	path	List<...>	Correspondence by regulator.
PUT	/correspondence/assign	LEGAL	AssignCorrespondenceRequest	...Response	Reassign correspondence.
PUT	/correspondence/updateStatus	LEGAL	UpdateCorrespondenceStatusRequest	...Response	Change correspondence status.
GET	/correspondence/getAll	LEGAL, ADMIN	—	List<...>	All correspondence.

Method	Path	Role	Request	Response	Description
POST	/penalties/add	LEGAL	PenaltyProceedingRequest	...Response	Add a penalty (OPEN).
GET	/penalties/getAll	LEGAL, CCO	—	List<...>	All penalties.
GET	/penalties/getByStatus?status=	LEGAL, CCO	query	List<...>	Filter penalties by status.
PUT	/penalties/updateStatus	LEGAL	UpdatePenaltyStatusRequest	...Response	Change status; sets paidDate when PAID.
GET	/audit-logs	ADMIN, AUDITOR	filters+Pageable	Page<AuditLogResponse>	Search audit logs.

Key logic

- **Return lifecycle** — create = DRAFT; the CO attaches a PDF and moves it to PENDING_CCO_APPROVAL; the CCO approves (status set toward SUBMITTED/APPROVED). Status is a validated String (checked against the enum), with no forced state-machine.
- **Document handling** — PDF stored under uploads/return-management-service/return-documents as <UUID>.pdf with metadata on the return; download streams with the original filename.
- **Dashboard** — pending approvals (returns in PENDING_CCO_APPROVAL) and open penalties (count + summed amount) computed from local tables.
- **Feign** — validates preparer (auth-service) and obligation/calendar/regulator (compliance-core); fail-open with fallbacks.

5.7 audit-analytics-service (Reviews, Evidence, Analytics)

Port: 8084

Main class: AuditAnalyticsServiceApplication — @SpringBootApplication, @EnableFeignClients, @EnableConfigurationProperties(JwtProperties)

Entities

Field	Type	Notes
AuditReview	—	reviewId(int), obligationId, auditorId, reviewPeriod, scope(String), findingsSummary, rating(AuditRating), reviewDate, status(AuditReviewStatus)
ControlEvidence	—	evidenceId(int), returnId, evidenceType, filePath(path only), uploadedById, uploadedDate, status(EvidenceStatus)
ComplianceReport	—	reportId(int), scope(ReportScope), metrics(TEXT/JSON), generatedDate
AuditLog	—	auditId(Long), userId, action, entityType, recordId, timestamp

Enums

AuditRating: SATISFACTORY, NEEDS_IMPROVEMENT, UNSATISFACTORY

AuditReviewStatus: DRAFT, FINAL

EvidenceStatus: UPLOADED, VERIFIED, REJECTED

ReportScope: OBLIGATION, RETURN, PENALTY, OVERALL

Endpoints

Method	Path	Role	Request	Response	Description
POST	/audit-reviews/add	AUDITOR	AuditReviewRequest	AuditReviewResponse	Create an audit review (DRAFT).
GET	/audit-reviews/myReviews/{auditorId}	AUDITOR	path	List<...>	Reviews by an auditor.
GET	/audit-reviews/getByRating?rating=	AUDITOR, CCO	query	List<...>	Reviews filtered by rating.
GET	/audit-reviews/getAll	ADMIN, CCO	—	List<...>	All reviews.

Method	Path	Role	Request	Response	Description
POST	/control-evidence/upload	AUDITOR, CO	ControlEvidenceRequest (JSON)	...Response	Register evidence (path reference; UPLOADED).
GET	/control-evidence/byReturn/{id}	authenticated	path	List<...>	Evidence for a return.
PUT	/control-evidence/updateStatus	AUDITOR	UpdateEvidenceStatusRequest	...Response	Verify / reject evidence.
POST	/analytics/reports/generate	CCO	GenerateReportRequest	ComplianceReportResponse	Build & persist report metrics (JSON) for a scope.
GET	/analytics/reports/getAll	CCO	—	List<...>	All reports.
GET	/analytics/reports/byScope/{scope}	CCO	path	List<...>	Reports by scope.
GET	/analytics/dashboard	CCO	—	CcoDashboardResponse	CCO dashboard aggregated via Feign (with degraded flag).
GET	/analytics/pending-returns	CCO	—	List<...Ref>	Returns pending CCO approval (from return-mgmt).
GET	/analytics/all-penalties	CCO	—	List<...Ref>	All penalties (from return-mgmt).
GET	/audit-logs	ADMIN, AUDITOR	filters+Pageable	Page<AuditLogResponse>	Search audit logs.

Key logic

- **Reports** — aggregate counts/amounts per scope by calling compliance-core and return-management over Feign, serialise to JSON in 'metrics', and include a 'degraded' flag if any upstream call fell back.
- **Dashboard** — mirrors the CCO dashboard but sourced over Feign, with circuit-breaker + retry and a degraded flag.
- **Evidence** — tracks a file-path reference (not bytes); status UPLOADED → VERIFIED / REJECTED.

5.8 notification-service (In-app Notifications)

Port: 8085

Main class: NotificationServiceApplication — @SpringBootApplication, @EnableFeignClients, @EnableConfigurationProperties(JwtProperties)

Entities & enums

Field	Type	Notes
Notification	—	notificationId(int), userId(int), message, category(NotificationCategory), status(NotificationStatus), createdDate
AuditLog	—	auditId(Long), userId, action, entityType, recordId, timestamp

NotificationCategory: OBLIGATION, DATA_COLLECTION, RETURN, CORRESPONDENCE, PENALTY, AUDIT

NotificationStatus: UNREAD, READ

Endpoints

Method	Path	Role	Request	Response	Description
GET	/notifications/my/{userId}	authenticated	path+Pageable	Page<...>	All notifications for a user.
GET	/notifications/unread/{userId}	authenticated	path	List<...>	Unread notifications.
GET	/notifications/count/{userId}	authenticated	path	UnreadCountResponse	Unread count.
PUT	/notifications/markRead/{id}	authenticated	path	...Response	Mark one notification read.
PUT	/notifications/markAllRead/{userId}	authenticated	path	List<...>	Mark all a user's notifications read.
POST	/notifications	authenticated	CreateNotificationRequest	...Response	Create a notification (UNREAD). Called service-to-service.

Method	Path	Role	Request	Response	Description
GET	/audit-logs	ADMIN, AUDITOR	filters+Pageable	Page<AuditLogResponse>	Search audit logs.

Notifications are created by other services (e.g. compliance-core) via Feign, propagating the end user's JWT. The target userId is supplied explicitly because the JWT carries email + role, not the numeric id.

6. End-to-End Business Flow

The typical lifecycle, and where each role acts:

- **Step 1 — Admin sets up the base** — create a Regulator, then an Obligation (assigning a CO), then a Deadline on the calendar (due date + data cut-off).
- **Step 2 — CO requests data** — the Compliance Officer creates a Data Request assigned to a Data Owner, linked to the deadline. The data owner is notified.
- **Step 3 — Data Owner provides & certifies** — the Data Owner uploads the required PDF (status → DATA PROVIDED) and then certifies it; certification automatically marks the request CERTIFIED / COMPLETED.
- **Step 4 — CO creates the return** — using the certified data, the CO creates a Regulatory Return, attaches the PDF, and moves it to PENDING CCO APPROVAL.
- **Step 5 — CCO reviews & approves** — the CCO opens the return, downloads and reviews the PDF, and approves (or rejects). Approved returns proceed to submission.
- **Step 6 — Supporting activities** — Auditors record audit reviews and ratings; Legal Officers record and track penalties; notifications are sent at each step.

7. Frontend Application

A single-page React app (frontend-monolith). It is a monolithic UI adapted to the microservices backend: every backend-specific concern (paths, role vocabulary, status vocabulary, auth headers) is isolated in one adapter, src/services/api.js, so the page components stay unchanged.

7.1 Stack & dev server

Stack: React 18.2, React DOM 18.2, React Router DOM 6.22, Axios 1.6, Vite 5, @vitejs/plugin-react 4.2

Dev server: Vite — host 127.0.0.1, port from PORT env (default 5173), strictPort

Entry: main.jsx mounts <App/> (StrictMode) and imports index.css; App.jsx wraps routes in <BrowserRouter>

7.2 Folder structure (src/)

main.jsx, App.jsx, index.css; components/ (Dropdown.jsx, DatePicker.jsx); routes/ (AppRoutes.jsx, RequireAuth.jsx); services/ (api.js, auth.js); pages/ grouped by role: user/, admin/, co/, dataowner/, legalofficer/, auditor/, cco/. There is no shared Navbar — each page renders its own role-specific <nav>; session state lives in localStorage (no React Context).

7.3 Routing & guards

AppRoutes.jsx defines every route; protected routes are wrapped in <RequireAuth roles={[...]}>. Selected routes:

Path	Component	Role
/ , /login	Login	Public
/register	Register	Public (self-register = ADMIN only)
/users	AllUsers	ADMIN
/admin/add-regulator, /all-regulators	AddRegulator, AllRegulators	ADMIN
/admin/add-obligation, /all-obligations	AddObligation, AllObligations	ADMIN
/admin/add-deadline, /calendar	AddDeadline, CalendarView	ADMIN
/co/create-return, /my-returns	CreateReturn, MyReturns	COMPLIANCE_OFFICER
/co/returns-by-status, /calendar	ReturnsByStatus, CalendarView	COMPLIANCE_OFFICER
/co/create-data-request, /view-data-requests	CreateDataRequest, ViewDataRequests	COMPLIANCE_OFFICER
/dataowner/my-requests, /add-certification	MyDataRequests, AddCertification	DATA_OWNER
/legal/add-penalty, /all-penalties	AddPenalty, AllPenalties	LEGAL_OFFICER
/auditor/add-review, /my-reviews	AddAuditReview, MyAuditReviews	INTERNAL_AUDITOR
/auditor/all-returns, /all-penalties, /all-data-requests, /all-obligations	read-only views	INTERNAL_AUDITOR
/cco/dashboard, /all-returns, /all-obligations	CCODashboard, AllReturns, AllObligations	CCO
/cco/calendar, /all-penalties, /all-audits	CalendarView, AllPenalties, AllAudits	CCO
/notifications	Notifications	Any authenticated

RequireAuth redirects to /login if not logged in or if the role is not allowed. isLoggedIn()/getRole() come from auth.js (they treat an expired token as logged-out).

7.4 Authentication (frontend)

Storage: localStorage keys: token, role, userId, userName

Login: calls loginUser(); stores token/role/userId/userName; redirects by role (ADMIN→/users, CCO→/cco/dashboard, CO→/co/my-returns, DATA_OWNER→/dataowner/my-requests, LEGAL→/legal/add-penalty, AUDITOR→/auditor/my-reviews)

auth.js: decodeJwt, isLoggedIn (checks exp), getRole, clearSession

7.5 API adapter (services/api.js)

Base URL: http://localhost:9080 (the API gateway)

Request interceptor: adds Authorization: Bearer <token> from localStorage

Response interceptor: on 401, clears the session and redirects to /login

Vocabulary translation (monolith UI ↔ backend)

Domain	UI value → backend value
Role	COMPLIANCE_OFFICER↔CO, INTERNAL_AUDITOR↔AUDITOR, LEGAL_OFFICER↔LEGAL (ADMIN/DATA_OWNER/CCO same)
Data-request status	SENT↔PENDING, DATA_PROVIDED↔IN_PROGRESS, CERTIFIED↔COMPLETED, REJECTED↔OVERDUE
Return status	UNDER_REVIEW↔IN_PROGRESS, ACKNOWLEDGED↔APPROVED, LATE↔REJECTED (DRAFT/PENDING_CCO_APPROVAL/SUBMITTED same)
Penalty status	APPEALING↔DISPUTED, CLOSED↔WAIVED (OPEN/PAID same)
Frequency	ANNUAL→ANNUALLY

Representative API functions (Function — endpoint — purpose)

Function	Endpoint	Purpose
loginUser	POST /auth/login (+ /users/me)	Authenticate; fetch profile; map role to long form
addUser / registerUser	POST /auth/register	Create a user (role mapped to short enum)
getAllUsers / getUsersByRole	GET /users, /users/role/{r}	List users (unwrap .content, cache for lookups)
updateUserStatus / deleteUser	PUT /users/status, DELETE /users	Activate/deactivate or delete a user
get*Notifications / markRead	GET/PUT /notifications/*	Fetch, count and mark notifications read
addRegulator / getAll / getActive / delete	/regulators/*	Regulator CRUD (soft-delete)
addObligation / getAll / getActive / byRegulator / myObligations / updateStatus	/obligations/*	Obligation management
addDeadline / upcoming / overdue / byObligation / updateStatus	/calendar/*	Compliance calendar
createDataRequest / getMyDataRequests / updateDataRequestStatus	/data-requests/*	Data-request lifecycle
uploadDataFile / downloadDataFile / getDataFileName	/data-requests/{id}/documents*	Data-owner PDF upload / download
addCertification	POST /certifications/add	Data owner certifies a request
createReturn / getMyReturns / getReturnsByStatus / updateReturnStatus	/returns/*	Return lifecycle
uploadReturnDocument / downloadReturnDocument	/returns/{id}/document*	CO attaches PDF; CCO downloads it
getPendingReturns / getCCODashboard	/returns/pendingApprovals, /analytics/dashboard	CCO approvals & metrics
addAuditReview / getMyReviews / getReviewsByRating / getAllAudits	/audit-reviews/*	Audit reviews

Function	Endpoint	Purpose
addPenalty / getAllPenalties / getPenaltiesByStatus / updatePenaltyStatus	/penalties/* (+ /correspondence/create)	Penalty management (auto-creates a correspondence)

7.6 Pages by role

Data Owner (the highlighted role)

- **MyDataRequests** — the data owner's inbox: lists assigned requests, provides a PDF-only file input (upload) for SENT/DATA_PROVIDED requests, and a status dropdown. This is the core data-submission screen.
- **AddCertification** — certify a request whose PDF is uploaded: pick a DATA_PROVIDED request, enter a certification statement and date, and submit (status becomes Certified).

Other roles (summary)

- **Admin** — AddRegulator / AllRegulators, AddObligation / AllObligations, AddDeadline, CalendarView, and AllUsers (user management: activate/deactivate/delete, role filter).
- **Compliance Officer** — CreateReturn (with optional PDF), MyReturns, ReturnsByStatus, CalendarView, CreateDataRequest, ViewDataRequests (download submitted data).
- **CCO** — CCODashboard (metrics + pending approvals with View PDF / Approve & Submit), AllReturns, AllObligations, CalendarView, AllPenalties, AllAudits.
- **Auditor** — AddAuditReview, MyAuditReviews, plus read-only AllReturns / AllPenalties / AllDataRequests / AllObligations.
- **Legal Officer** — AddPenalty (auto-fills from overdue deadlines), AllPenalties (status filter, paid-date modal, locked PAID/CLOSED rows).
- **User (shared)** — Login, Register (admin adds users / self-register admin), Notifications (feed with categories, filter, mark-read).

7.7 Shared components & styling

- **Dropdown.jsx** — custom replacement for <select>; the menu is a normal DOM element locked to the field width (long labels wrap, list scrolls), so it never overflows the card. Closes on outside-click / Escape.
- **DatePicker.jsx** — custom replacement for <input type=date>; a month-grid calendar that emits yyyy-mm-dd (backend contract) and displays dd-mm-yyyy, constrained to the field width.
- **Styling** — a single index.css theme ('Classic UI', earthy palette: olive nav, terracotta primary, cream background), serif headings, badge/table/card/stat components, matching styles for the custom dropdown/date-picker, and responsive breakpoints (tables scroll, grids collapse, nav stacks on small screens).

8. Notes & Current Limitations

Honest, factual notes about the current state (useful for maintenance and future work):

- Configuration is development-grade: plaintext JWT secret and DB credentials (root/root) in the config repo, ddl-auto=update, auto-created databases. Externalise secrets and use migrations for production.
- The gateway's public-path check uses substring matching, so a path merely containing '/auth/' etc. would bypass the JWT gate — worth tightening to prefix-only matching.
- User validation across services is fail-open because auth-service does not yet expose GET /users/{id}/exists; the Feign fallbacks return 'exists = true'.
- Certification status is always CERTIFIED in code (PENDING/REJECTED unused); DocumentStatus.FORWARDED and AuditReviewStatus.FINAL are defined but not currently set.
- Return status is a validated String with no enforced state-machine — any valid enum value can be set via updateStatus.
- A couple of frontend view fields have known mismatches (e.g. cco/AllReturns passes calendarId to the return-PDF download; cco/AllAudits reads auditedById/findings vs the auditorId/findingsSummary that reviews are created with).